

LLM-Enhanced Representations for Task Routing

MTP 2 Presentation - 2026

Gaurav Kumar
Guide: Prof. Sunita Sarawagi
Department of CSE, IIT Bombay

What is Task Routing?

Selecting the right item from a large pool for a given query:

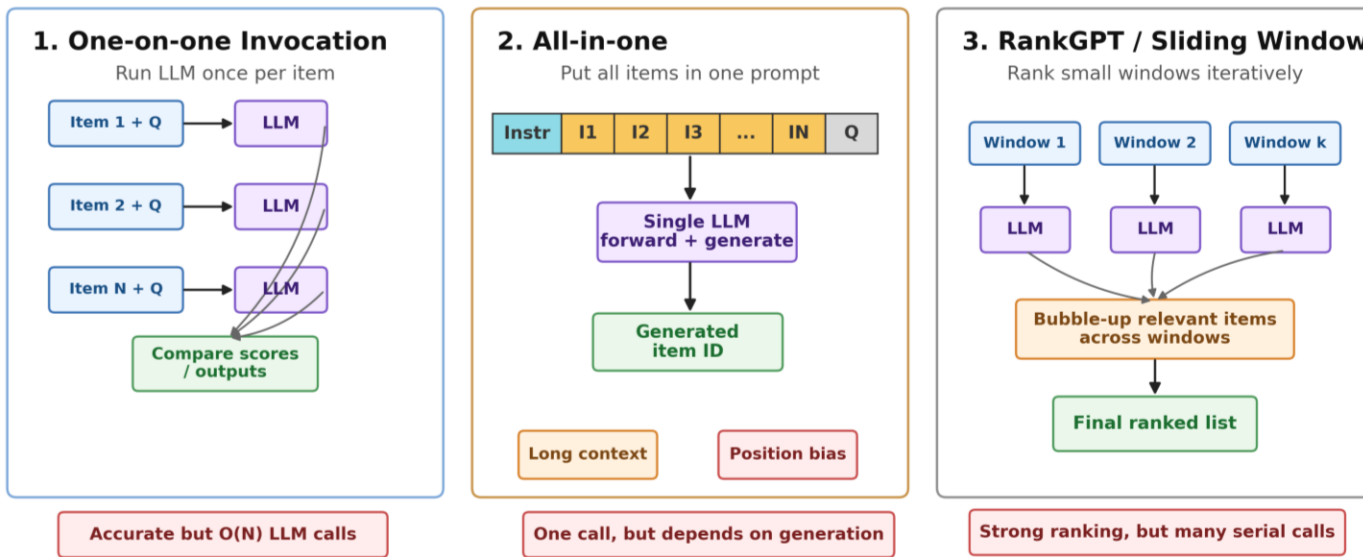
- Database selection (which schema can answer this query?)
- Tool selection (which tool or API needs to be called?)
- Document selection (which document supports/answer the query?)

Why LLMs?

- world knowledge
- reasoning abilities
- semantic and context understanding

LLMs can solve routing tasks very well **when prompted correctly.**

Traditional LLM Routing Approaches



Problem with generation

- Latency grows with pool size
- Positional bias affects ranking

We want to use what the LLM already computes.

Insight: LLM Internal Signals Are Rich

LLMs produce signals that can encode both semantics and relevance:

- Hidden states
- KV-caches
- Attention matrix

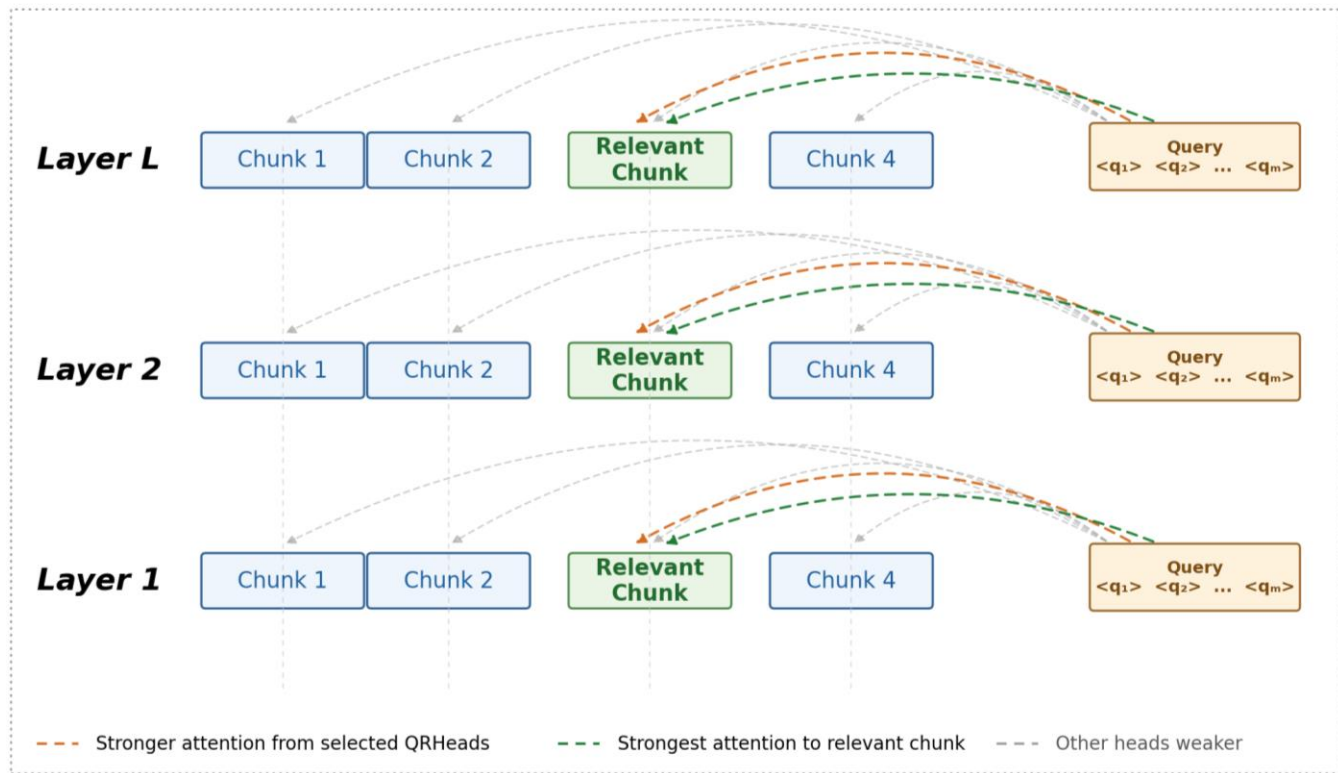
Cheaper to extract than generation!

A Different Signal from LLM: Retrieval Patterns in Heads

- Recent papers showcasing evidence of retrieval heads:
 - Retrieval Head (Wu et al., 2024): identifies “retrieval heads” via copy-paste logic
 - QRHead (Zhang et al., 2025): learns retrieval heads on NQ dataset

Collectively, these works demonstrate that attention heads can encode retrieval signals.

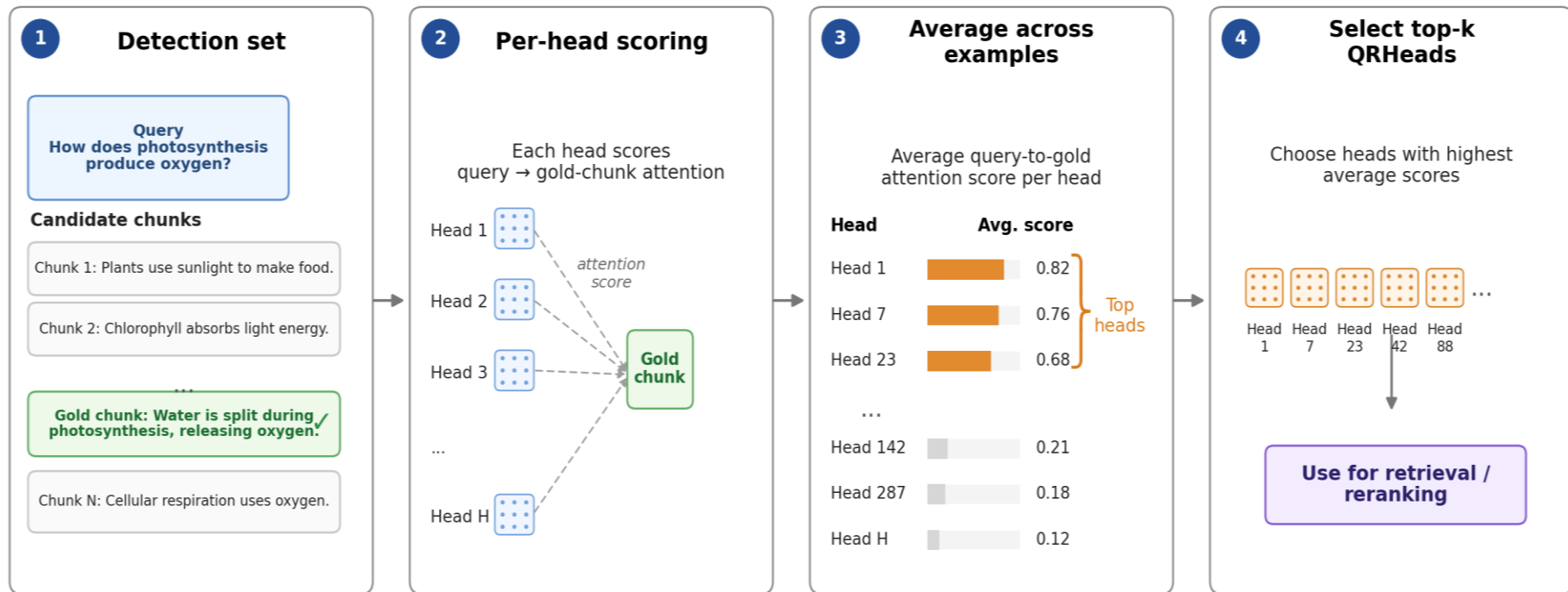
Query-Focused Retrieval Heads(QRHead)



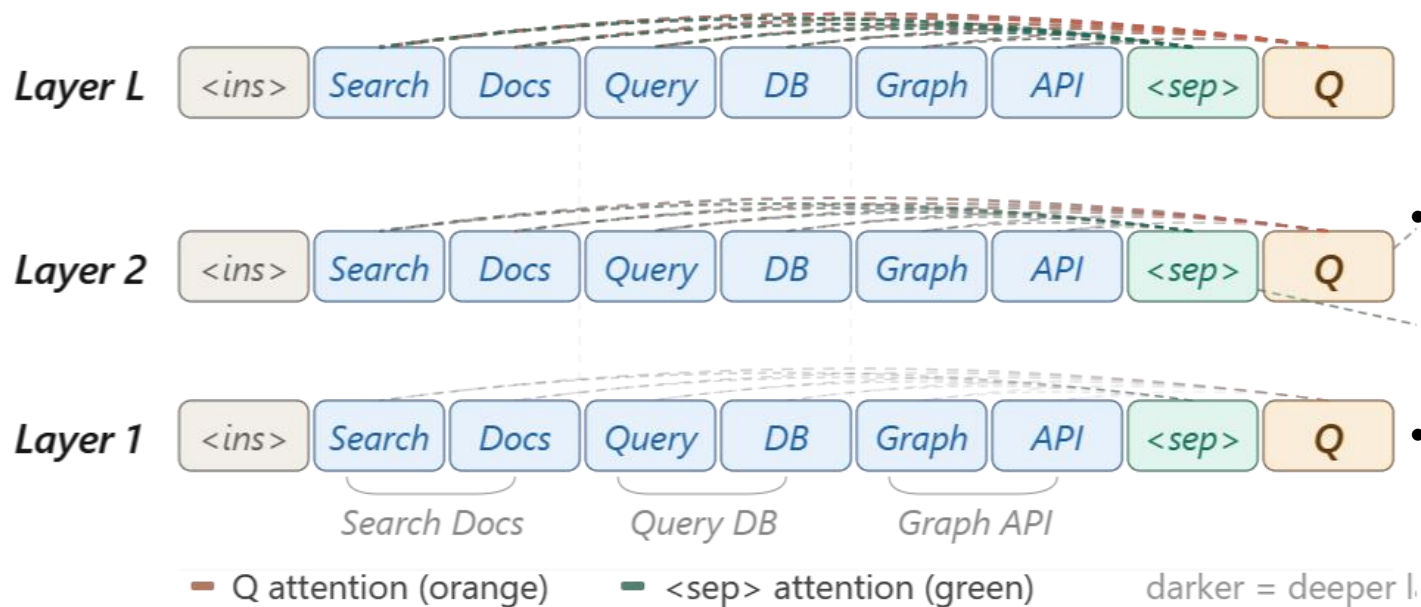
- Idea: a good retrieval head should place more attention from query tokens onto gold/relevant passages.
- QRHead looks at query $\rightarrow$ relevant context attention.
- Only a small detection set is needed.



QRHead: Head Detection



Retrieval Heads

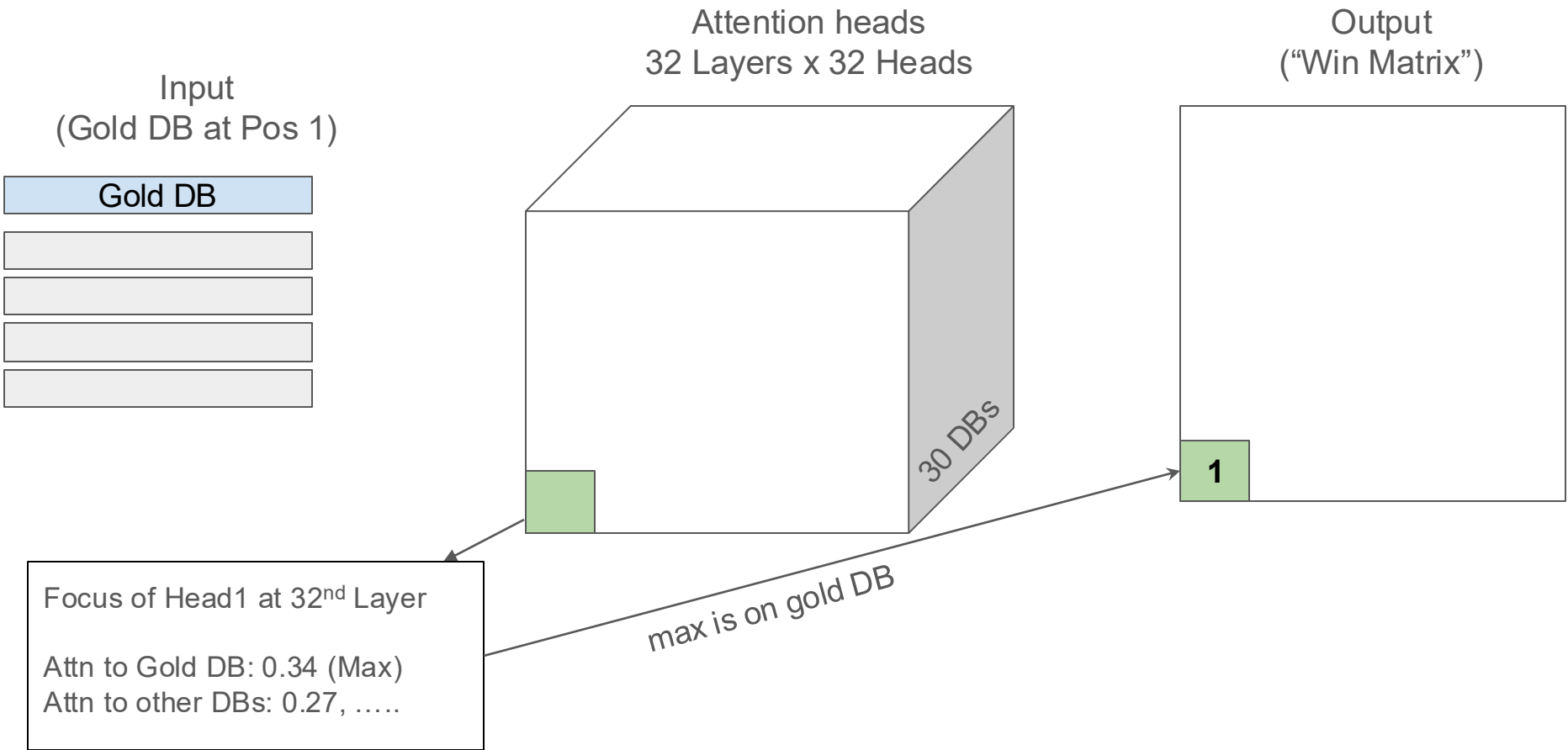


- Expect more attention on relevant tool names
- Not all attention heads follow that pattern
- Subset of heads take the role of linking to distant relevant tokens: called retriever heads

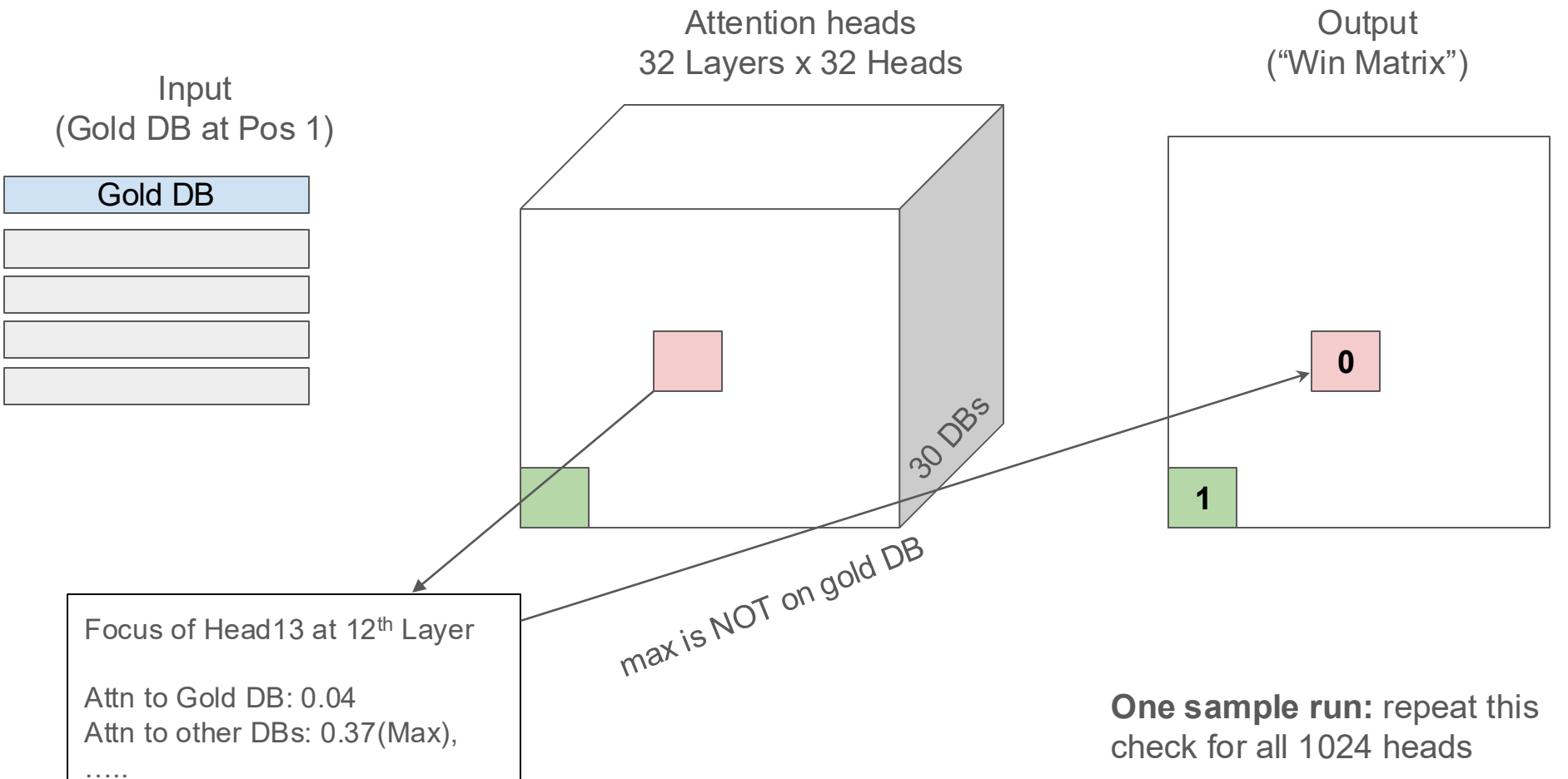
Mechanistic evidence capturing the retrieval heads

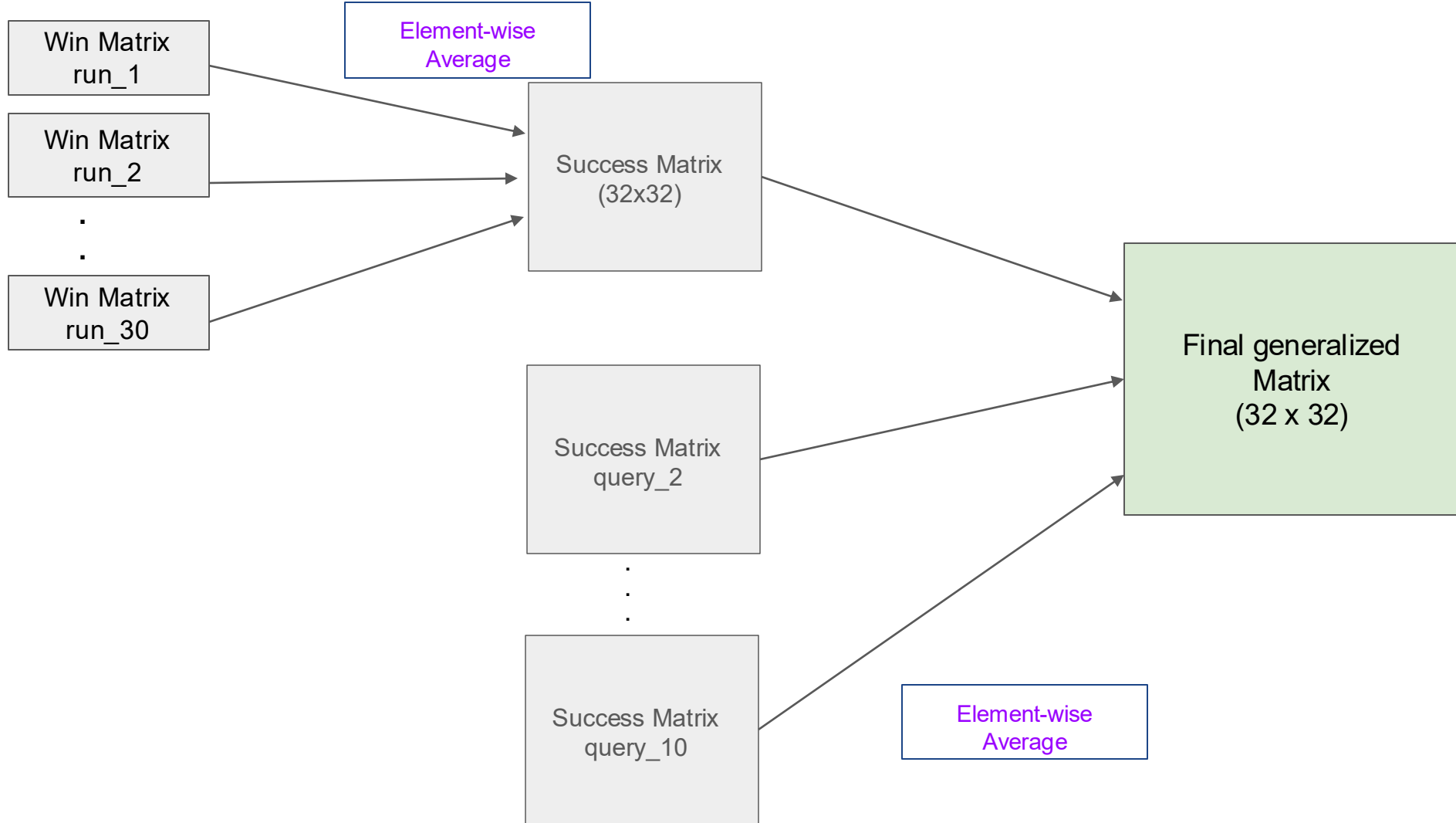
- We conducted controlled experiment to analyse the behaviour of retrieval heads
- For each query, one **gold database** is placed among multiple distractor databases
- For every layer and head, we measure whether its maximum attention falls on the gold database
- Heads with consistently high success are treated as evidence of retrieval-oriented behavior

Mechanistic evidence

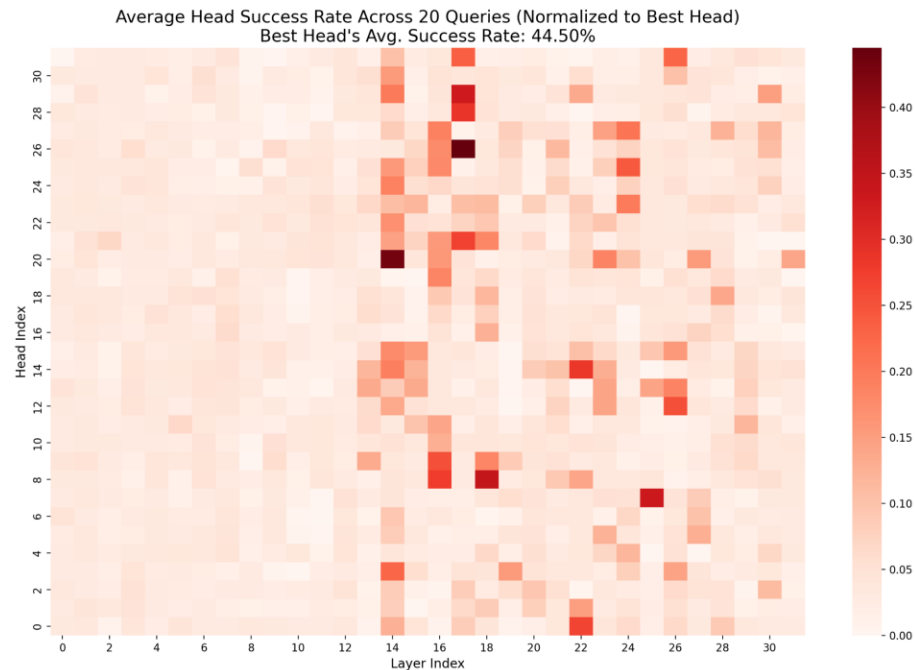
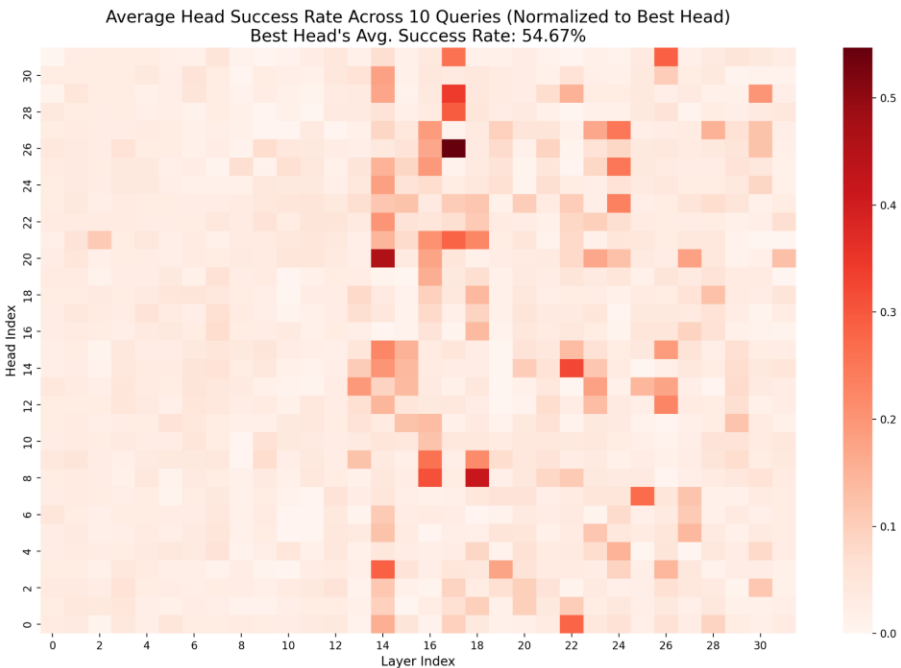


Mechanistic evidence





Aggregated Result for N = 10 & N = 20 Queries



LLMs already encode retrieval signals inside their attention heads.

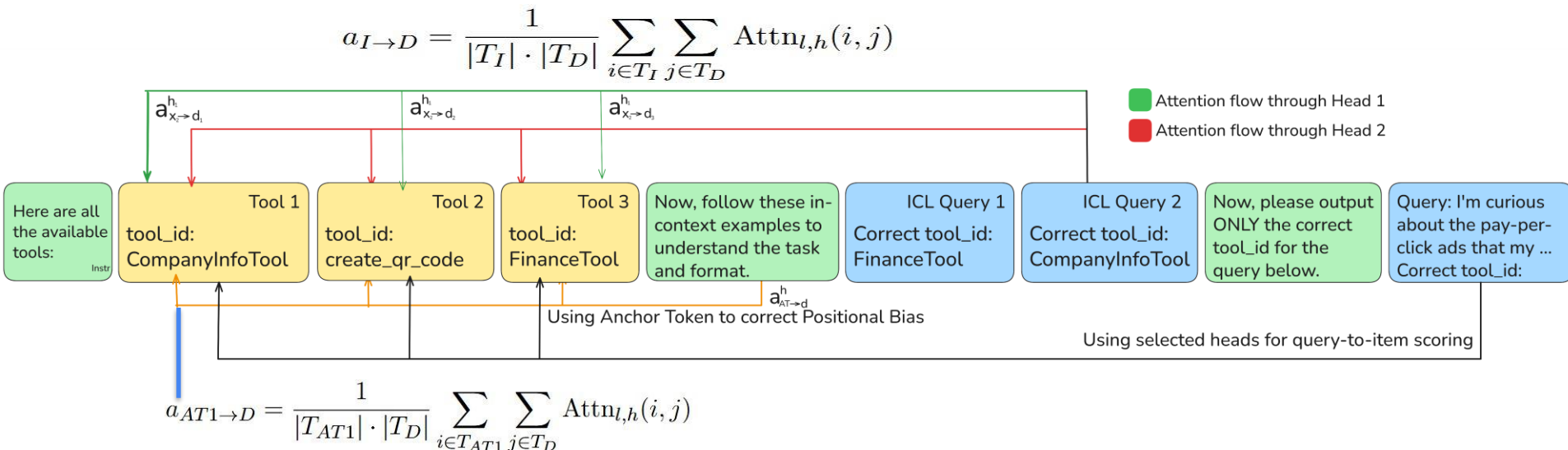
Motivation for OLR Heads

- QRHeads showed a small subset of attention heads can act as retrieval heads
 - But selecting one fixed set of heads offline may not generalize across tasks, prompts, or changing candidate lists
- OLR Heads solves this by selecting important heads based on few ICL examples in the prompt

Motivation for OLR Heads

- Attention scores are also position biased: an item may receive high attention because of where it appears, not because it is relevant
 - Existing correction technique used in QRHead require an extra call
- OLR Heads solves this online by using anchor tokens

OLR Heads Internals: Head Selection (online)



$$\tilde{a}_{I \rightarrow D} = a_{I \rightarrow D} - a_{AT1 \rightarrow D} \quad \text{Position Corrected with AT1}$$

$$A_{l,h}(q, d^*) = \sum_I \tilde{a}_{I \rightarrow d^*}$$

$$\mathcal{H}_{\text{top}} = \text{TopK}(\{A_{l,h}(q, d^*) \mid l \in [1, L], h \in [1, H]\})$$

OLR Heads Internals: Inference

$$a_{q_t \rightarrow D_i} = \frac{1}{|T_{q_t}| \cdot |T_{D_i}|} \sum_{m \in T_{q_t}} \sum_{n \in T_{D_i}} \text{Attn}_{l,h}(m, n)$$

$$\tilde{a}_{q_t \rightarrow D_i} = a_{q_t \rightarrow D_i} - a_{AT1 \rightarrow D_i}$$

$$S(q_t, D_i) = \sum_{(l,h) \in \mathcal{H}_{\text{top}}} \tilde{a}_{q_t \rightarrow D_i}$$

Effect of position correction on test query

Q: What is the height of the tallest player? Indicate his name.

gold db: european_football_2; position: 0

PC0: ['california_schools', 'european_football_2']

scores: [0.00258, 0.0024575]

PC1: ['european_football_2', 'california_schools']

scores: [0.0020403, 0.001838]

Q: In the branch where the second-highest number of crimes were committed in 1995 occurred, how many male clients are there?

gold db: financial; position: 9

PC0: ['debit_card_specializing', 'financial']

scores: [0.0052621, 0.0050883]

PC1: ['financial', 'debit_card_specializing']

scores: [0.0048956, 0.0045865]

OLR Heads

<|start_header_id|>user<|end_header_id|> Here are some paragraphs:

[1] Document Title
Document Description

[2] Document Title
Document Description

[3] Document Title
Document Description

Please refer to the relevant paragraphs using their numbers (e.g., [1], [2], etc.) when indicating your answer.
Now, follow these in-context examples below to understand the task and output format.

Query: icl_question_#1
Relevant Document number: [2]

Query: icl_question_#2
Relevant Document number: [21]

Now, please find which paragraph(s) contain information relevant to the following query:

Query: user_query
Relevant Document number:<|eot_id|>
<|start_header_id|>assistant<|end_header_id|>

→ anchor text(AT1)

→ anchor text(AT2)

← test query(Q)

Benchmarked Datasets

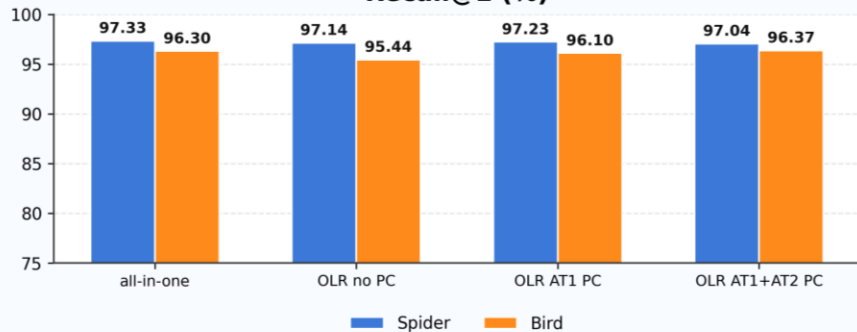
Fixed selection pool datasets

- Spider Dataset (1024 queries/20 databases)
- Bird Dataset (1534 queries/11 databases)
- ToolE Dataset (21k queries/199 tools)

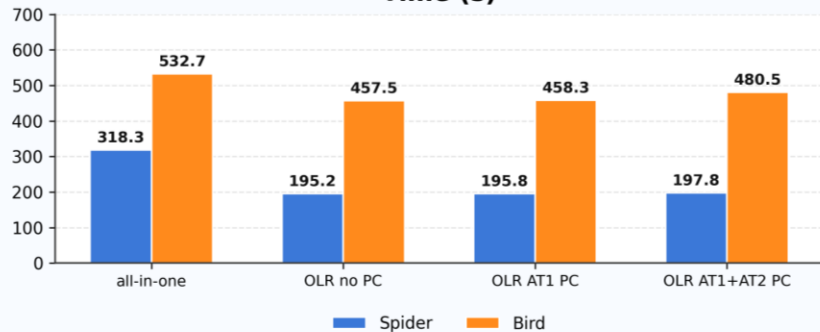
Results on Database selection:

Llama-3.1-8B-Instruct

Recall@1 (%)

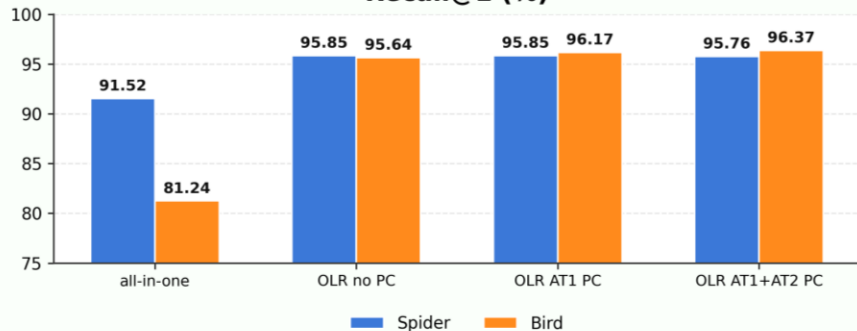


Time (s)

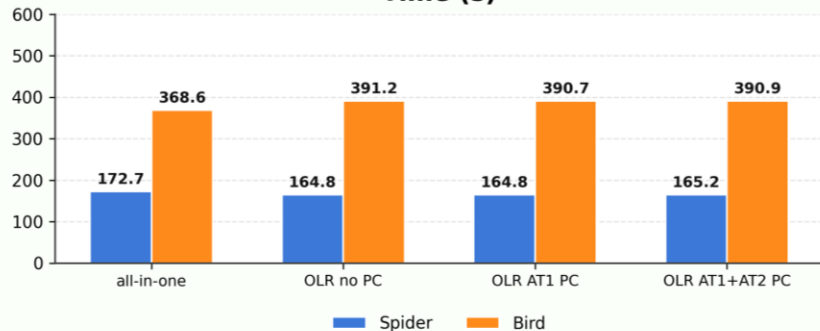


Qwen-2.5-7B-Instruct

Recall@1 (%)

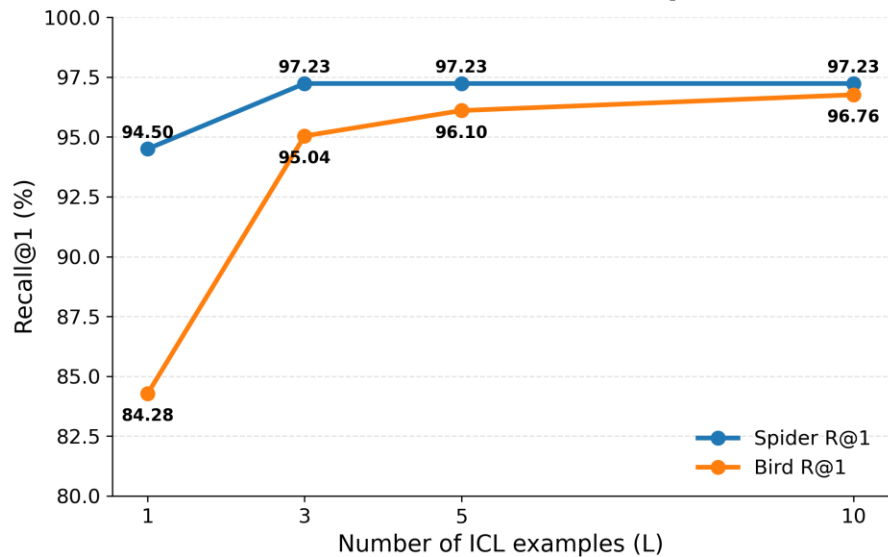


Time (s)

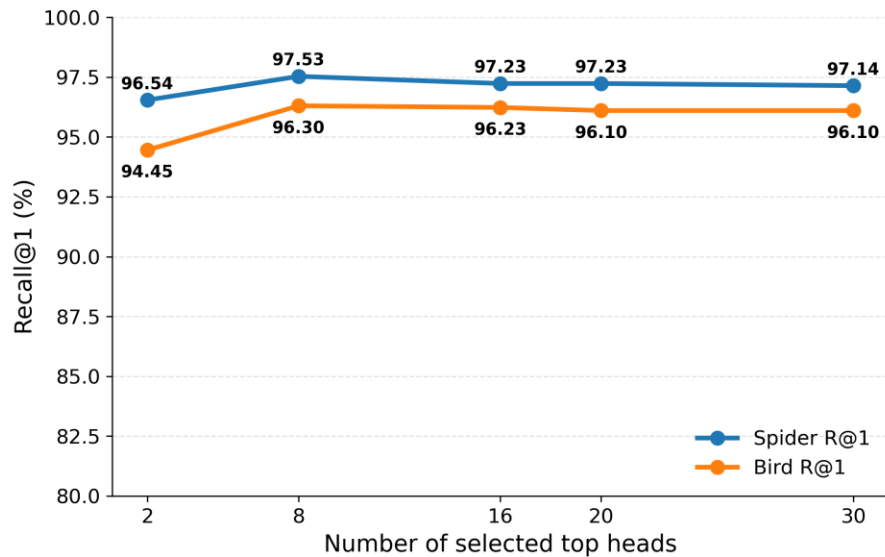


Ablation Study

Effect of More ICL Examples



Effect of Selected Heads

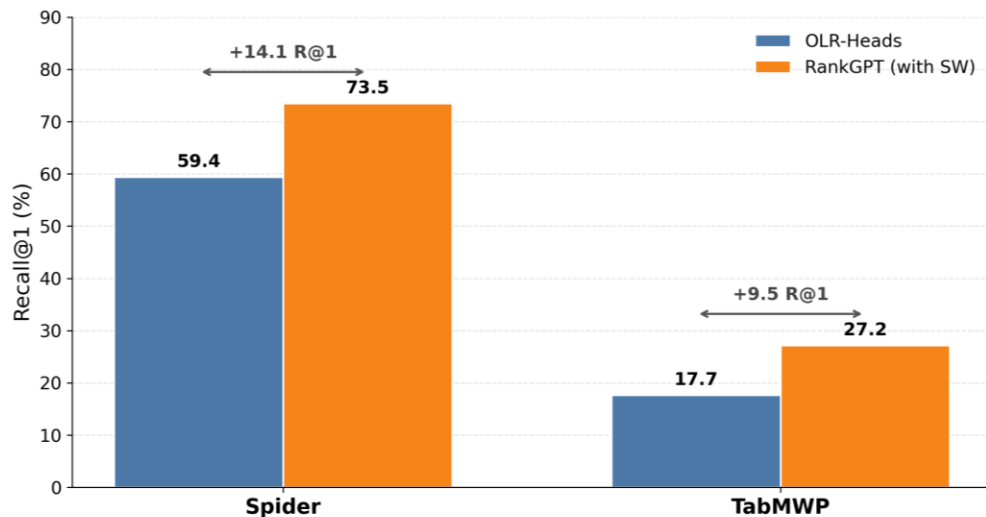


Results on Databases and Tool Selection

Method	ToolE	Spider	Bird	APIbank	Tooleyes	Algebra	Tabmwp
Prompt len	5k	42k	35k	25k	28k	64k	54k
Base LM: Llama-3.1-8B-Instruct							
Direct Gen.	62.2	53.3	57.3	75.3	58.8	41.1	19.0
ICR	51.3	63.4	66.2	64.7	61.3	35.2	13.0
QRHEAD	53.9	67.3	71.4	68.2	62.5	51.7	17.0
RankGPT(with SW)	59.3	60.9	53.1	72.9	51.3	57.2	13.0
RankGPT(without SW)	15.4	6.4	11.1	48.2	13.7	0.4	0.6
OLR-heads ^{sim}	62.5	67.0	73.4	77.6	68.8	60.2	19.0
Base LM: Qwen-2.5-7B-Instruct							
Direct Gen.	55.5	42.6	48.1	69.4	42.5	11.9	4.8
ICR	44.5	31.7	27.7	41.2	52.5	11.0	8.1
QRHEAD	51.3	51.2	38.9	49.4	58.8	15.3	17.7
RankGPT(with SW)	67.4	73.5	66.6	80.0	65.0	77.1	27.2
OLR-heads ^{sim}	59.4	59.4	65.4	76.5	55.0	52.1	17.7
Base LM: Qwen-2.5-7B-Instruct-1M							
Direct Gen.	63.1	52.9	58.5	77.6	48.8	27.5	8.8
ICR	48.7	54.5	65.1	49.4	60.0	39.4	15.6
QRHEAD	52.5	63.7	67.8	60.0	62.5	47.5	18.4
RankGPT(with SW)	67.4	74.5	70.1	82.4	67.5	78.8	25.9
RankGPT(constrained SW)	52.1	68.2	68.1	51.7	61.3	36.4	17.0
RankGPT(without SW)	37.1	37.8	41.9	51.7	42.5	30.5	6.8
OLR-heads ^{sim}	62.9	69.0	71.6	80.0	62.5	58.5	22.5

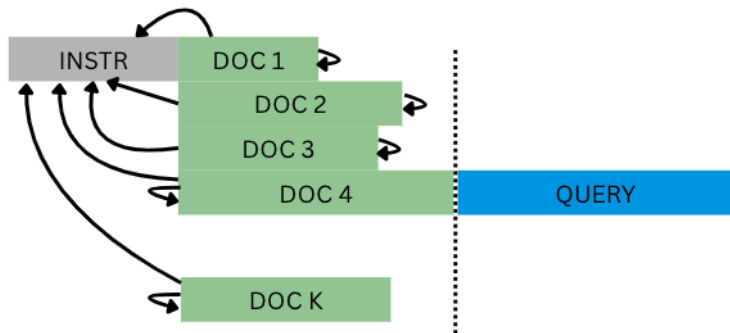
Limitations

- We saw that OLR Heads performs well enough but it is still worse than RankGPT
- We explore some other techniques, to catch the accuracy as good as RankGPT, but with low latency



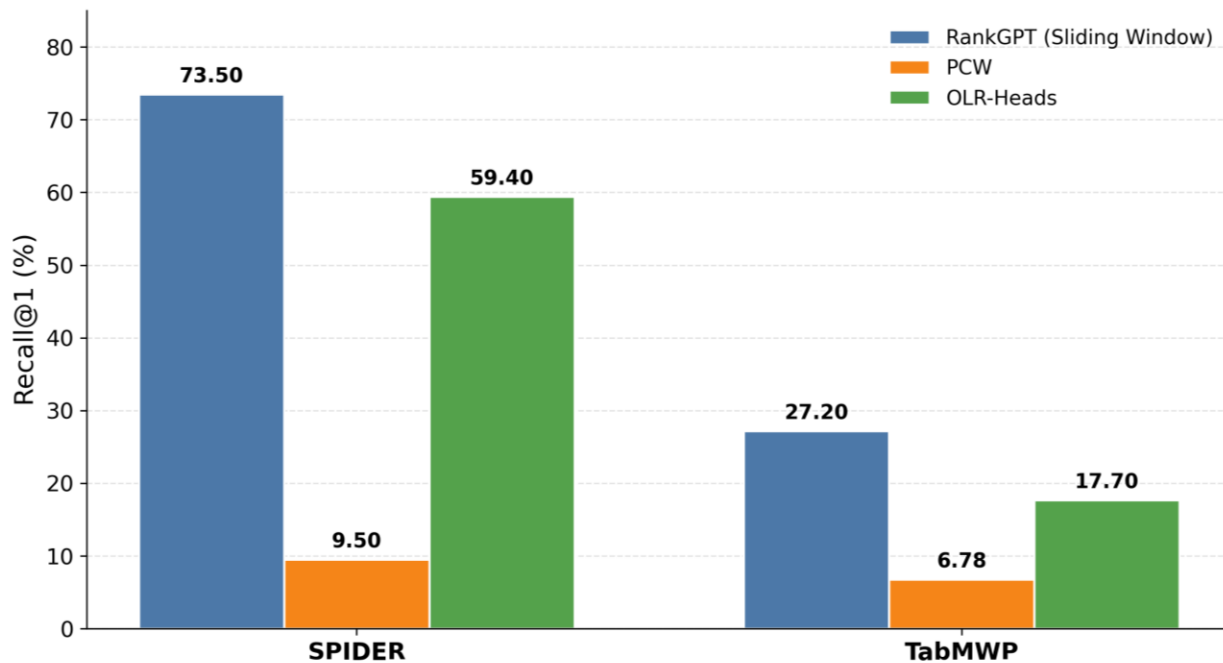
Results on Qwen 2.5-7B-Instruct

Optimisation 1: Parallel Context Window (PCW)



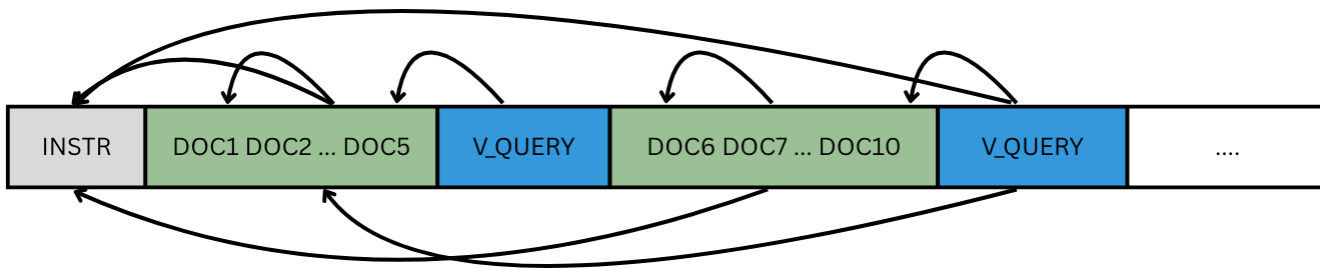
- Method: Each document is independently encoded with the INSTR, query is placed at $\max_len(docs)$ and score the documents
- Pros: Reusable KV cache, very fast inference
- Cons: Independently encoding the documents fails catastrophically for reranking tasks

Results on SPIDER and TabMWP using PCW



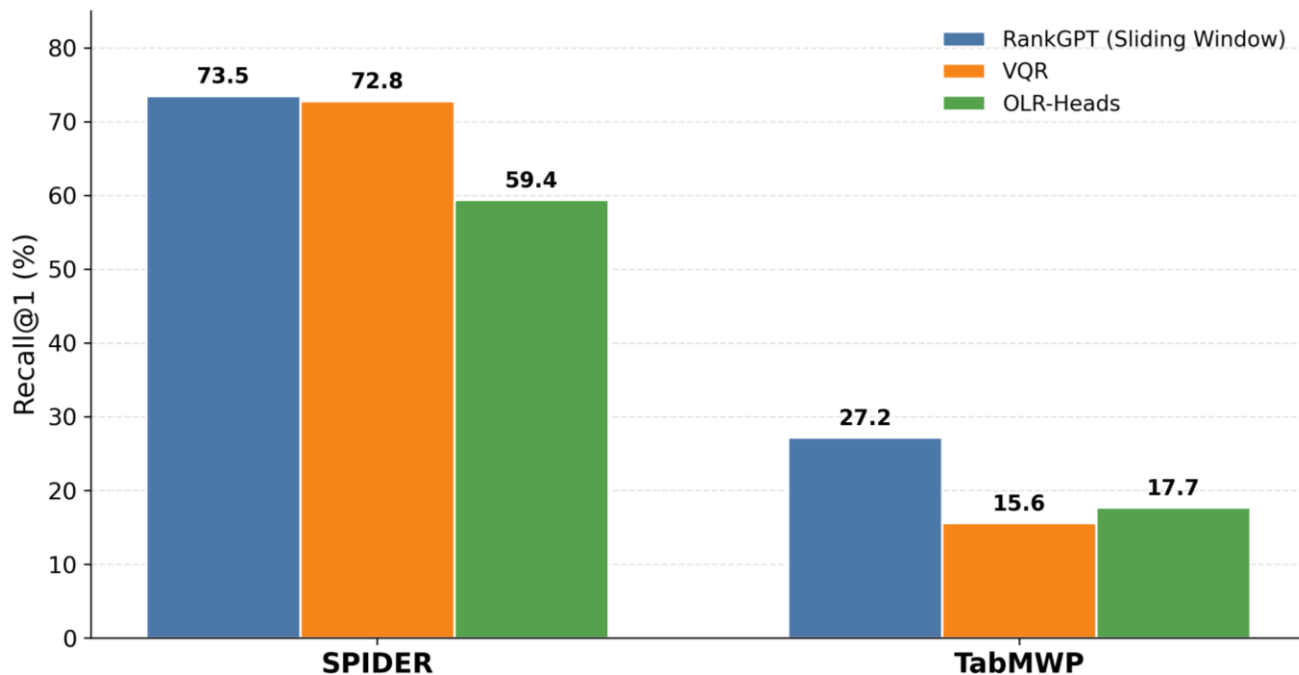
Retrieval performance using PCW on Qwen 2.5-7B Instruct

Optimisation 2: Virtual Query Repeaters (VQR)



- Method: Repeater copies inserted at fixed intervals (K) virtually
- Scoring: Aggregates attention mass from all virtual repeaters to rank candidates
- Pros: KV cache reuse, Re-anchors the query to maintain focus across long contexts

Results on SPIDER and TabMWP



Retrieval performance using VQR on Qwen-2.5-7B Instruct

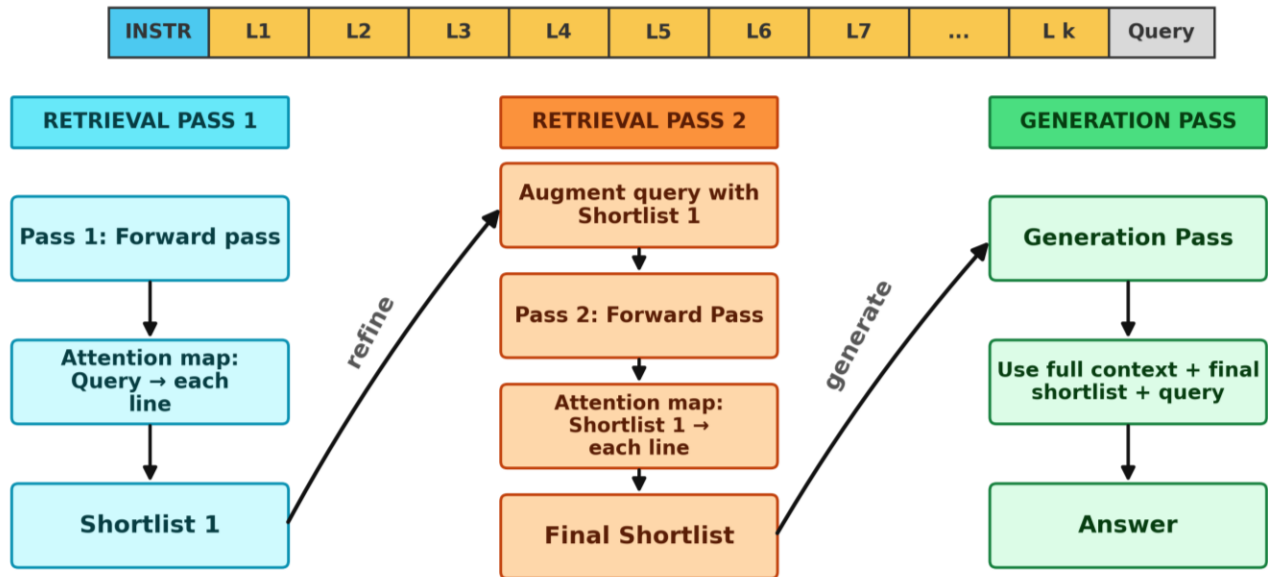
Limitations of reasoning systems

- Reasoning is expensive
 - A large number of LLM calls is done for single queries
- As context increases, performance degrades
 - Particularly for cost effective smaller models

Solution: Do not always generate. Directly retrieve from long context

- Wasteful to repeatedly encode the data subsets from LLM

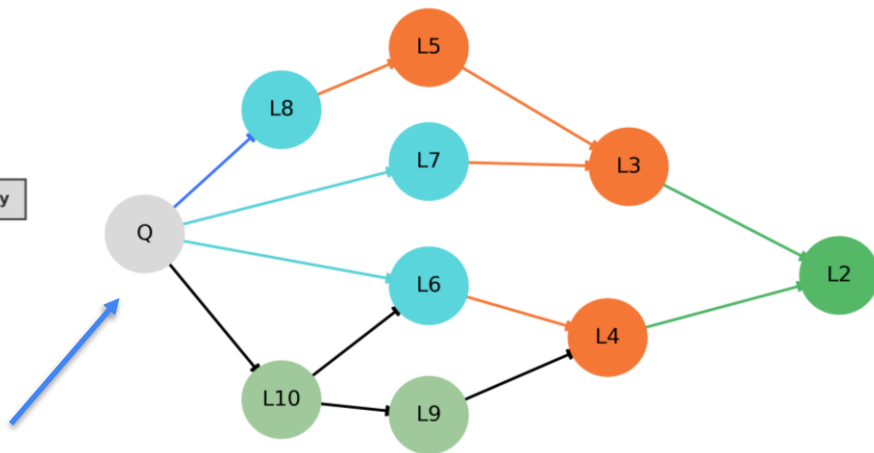
Improvement 1:



Improvement 2: Reasoning by Attention Graph Walk

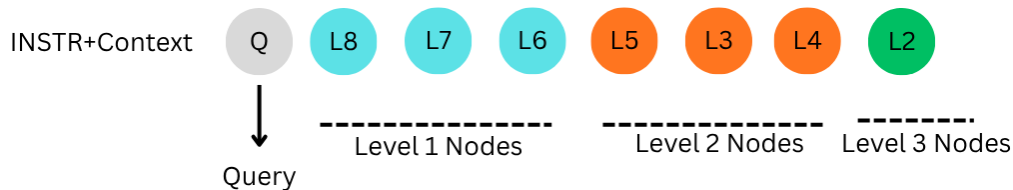


Step 1: Run a forward pass and get the attention map, using this attention map make attention graph and traverse to certain depth

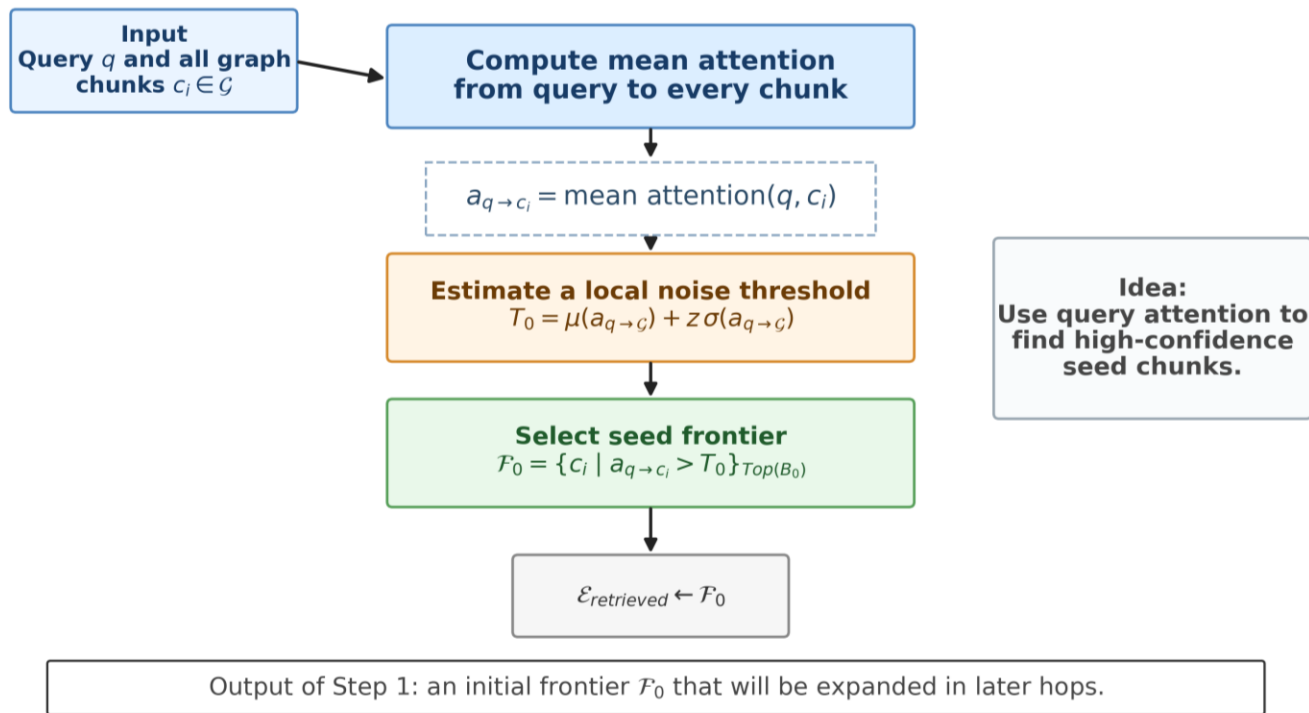


Step 2: After traversing the graph, get the shortlist and append it to the context and generate again to the answer

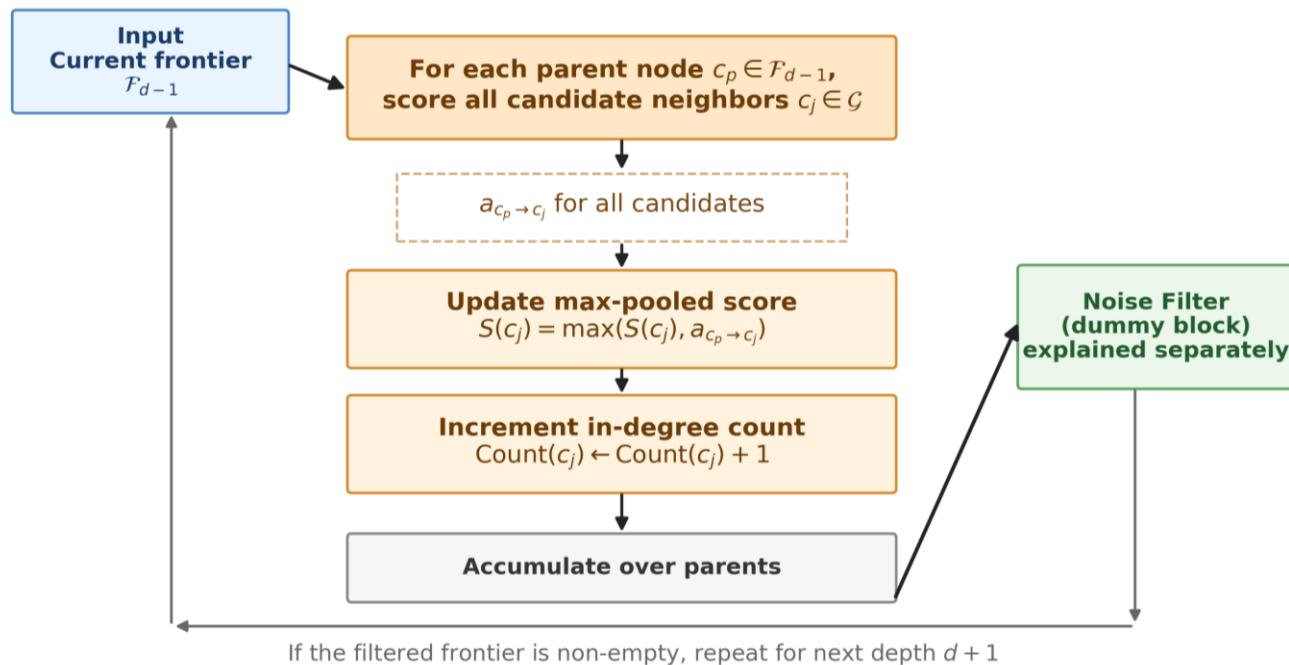
Total: 2 passes(1 retrieval and 1 generation)



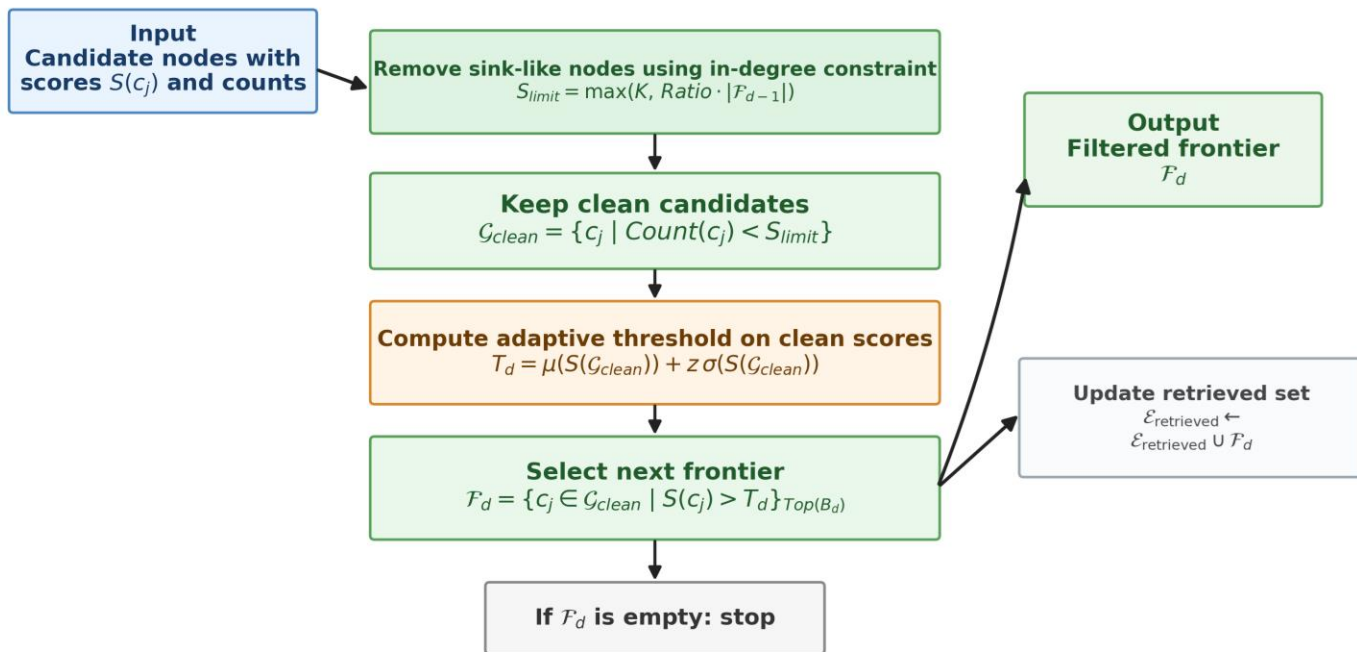
Attention Graph Walk Method: Step 1



Attention Graph Walk Method: Step 2



Attention Graph Walk Method: Step 3



Main idea: first clean the candidate pool, then threshold the clean scores to get the next frontier.

Sample Query from GraphWalks

<|im_start|>system\nYou will be given a graph as a list of directed edges.
All nodes are at least degree 1.

....
<|im_end|>\n

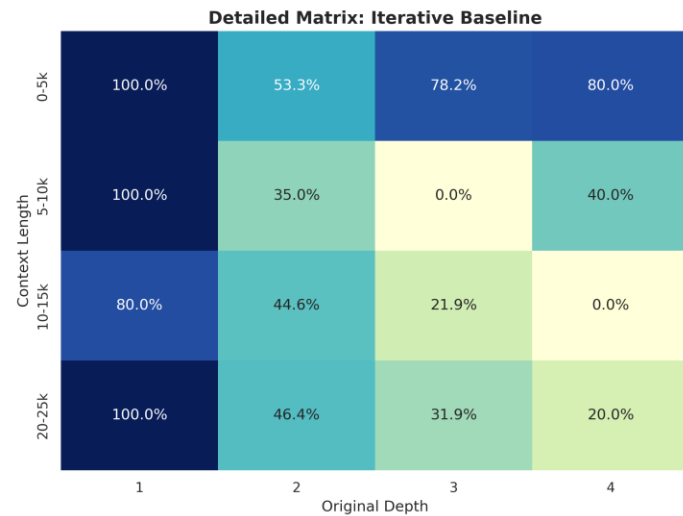
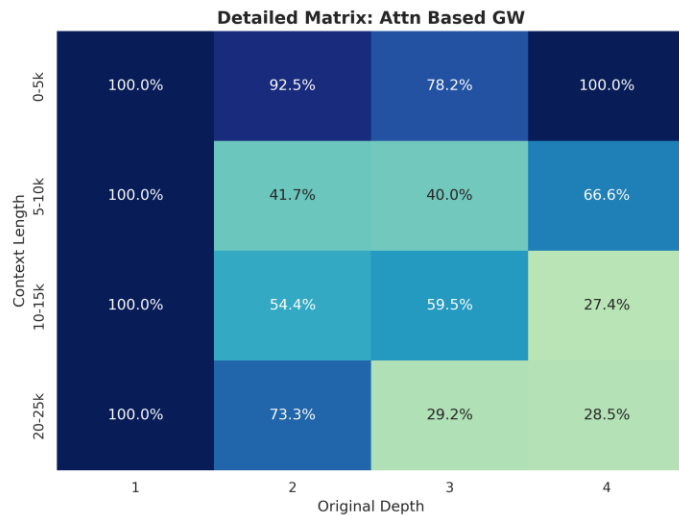
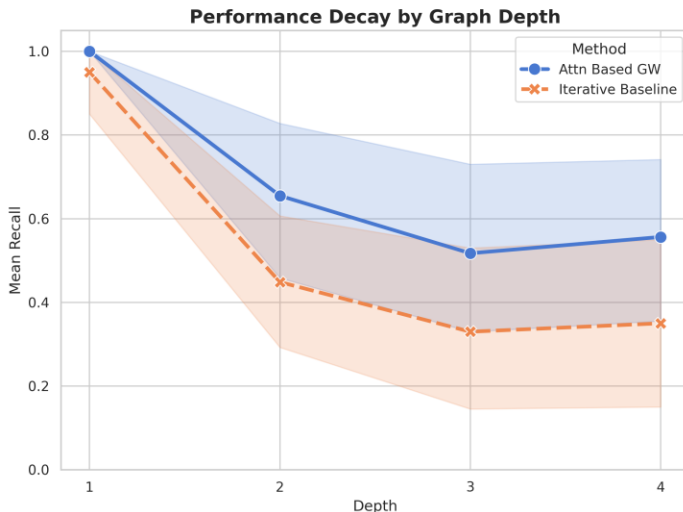
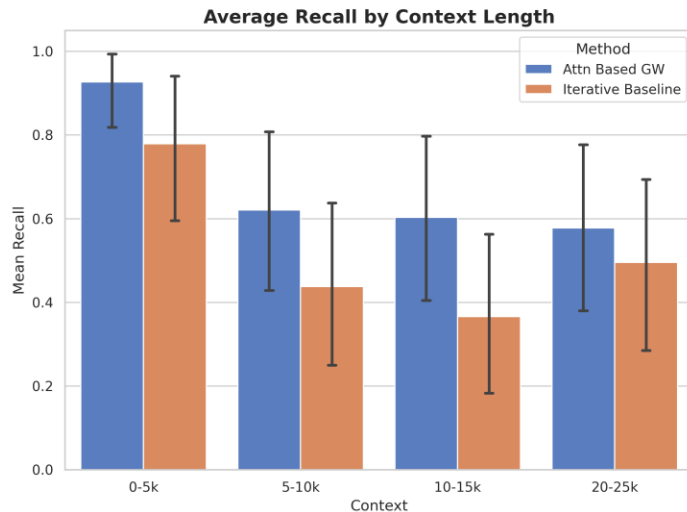
<|im_start|>user\nThe graph has the following edges:\n\ncfcd208495 -> f033ab37c3\n\ncfcd208495 -> d1fe173d08\n\ncfcd208495 -> 98dce83da5\n\ncfcd208495 -> ad61ab1432\n\ncfcd208495 -> ad61ab1432\n\nc4ca4238a0 -> fe9fc289c3\n\nc4ca4238a0 -> a3f390d88e\n\n....

\n\n**Operation:** Perform a BFS from node c20ad4d76f and return only the nodes at exactly depth 2(not nodes at intermediate depths)\n. You should reason through the operation step by step, and then return the set of nodes that the operation results in. Return your final answer as a list of nodes in the very last line of your response. For example, if the operation returns the set of nodes [node1, node2, node3], your response should be:\nFinal Answer: [node1, node2, node3]\nIf the operation returns the empty set, your response should be:\nFinal Answer:

[]<|im_end|>\n<|im_start|>assistant\n<think>\n\n"

Results on GraphWalk on Qwen 3B

Comparison: Attn Based GW vs Iterative Baseline



Conclusion

- Harness LLM internal representations rather than generation.
 - internal signals (attention, hidden states, KV-caches) are lightweight, interpretable, and could scale to longer contexts than token-by-token decoding.
- Retrieval signal is sparse.
 - Prior work and our experiments reveal that only a small subset of attention heads consistently encodes retrieval signals.
- OLR-heads
 - Dynamic, training-free retrieval-head selection
 - Fast positional correction
 - Single forward-pass scoring over selected heads
- Attention based multi-hop query answering
 - Navigates the attention graph to uncover multi hop reasoning relationships
 - Enable extraction of relationships in single pass

Thank You